Trabajo Práctico N.º 8 · Food Store en la feria

La base de la tablet (SQLite) y el cierre de la capa de datos · Ciclo Lectivo 2026 · Comisión 2PRO3

Integrantes: Belén Calvo, Elías Tello, Hernán González, Bruno Fiouchetta

Repositorio: github.com/belcalvo93/BaseDatos2-MongoDB · carpeta `food-store/`

> **Regla de oro: el móvil propone, el servidor dispone.** La tablet (SQLite) solo consulta una copia del catálogo y arma borradores sin señal. Precio, stock y ventas los decide únicamente PostgreSQL. `SIMULADO_APROBADO` significa "pasaría las validaciones", **no es una venta**, y ningún script de la simulación escribe en PostgreSQL. Todo se hizo sobre `bd2_tp8`, una copia de `bd2_tp3` con datos inventados.

Consigna 1.a · Qué datos viajan a la tablet

docs/1a_datos_que_viajan.md · postgresql/10_snapshot_catalogo.sql

Dato en PostgreSQL	¿Viaja?	Cómo llega a la tablet	Por qué
`producto.id_producto`	Sí	`producto_id`	El mismo id que en el servidor, para comparar en el semáforo.
`producto.nombre`	Sí	`nombre`	El vendedor tiene que ver qué vende.
`producto.precio` (NUMERIC)	Sí, convertido	`precio_centavos` (INTEGER)	Para mostrar el catálogo y detectar cambios de precio; en centavos las cuentas son exactas.
`producto.stock`	Sí, como copia	`stock_visto`	Solo orientativo: el stock real lo decide el servidor.
`producto.activo` y `categoria.activo`	Sí, convertidos	`eliminado` (lápida) y `disponible`	La baja viaja como dato: el producto sale del catálogo pero la fila queda. Cuenta la baja del producto **o** de su categoría (Congelados está dada de baja: 10.048 lápidas).
—	Se agrega	`capturado_en`	Fecha de la copia: dice qué tan vieja es la lista.
`descripcion`, `id_categoria`, `created_at`	No	—	No hacen falta para vender; la tablet tiene poco espacio.
`cliente` (nombre, apellido, email, teléfono)	No	—	Datos personales: no hacen falta para vender y la tablet se puede perder.
`pedido` y `detalle_pedido`	No	—	Solo el servidor confirma y numera ventas.
Usuarios, contraseñas, medios de pago	No	—	Riesgo sin beneficio; SQLite no tiene usuarios ni permisos.

A la tablet viaja solo lo necesario para mostrar el catálogo y armar borradores sin señal. Los datos personales no viajan porque la Ley 25.326 pide datos "adecuados, pertinentes y no excesivos" (art. 4) y que se cuide su seguridad (art. 9): en un dispositivo que se puede perder y que no tiene usuarios ni permisos, guardar poco es la principal medida de seguridad. **La fuente de verdad es PostgreSQL**: lo que guarda la tablet es una copia con fecha que puede estar vieja.

Consigna 2.c · Los casos C2 a C6

movil/casos_sqlite.sql · postgresql/casos_servidor.sql · evidencias/

Flujo de cada caso: (1) borrador en SQLite dentro de `BEGIN … COMMIT` → (2) si el caso lo pide, cambio manual en `bd2_tp8` → (3) `\copy` de una instantánea nueva → (4) `.read movil/importar_catalogo.sql` (upsert al caché) → (5) `.read movil/semaforo.sql` → (6) lectura de `cola_sync`. Los productos fijos y sus datos iniciales en el servidor:

id	Producto	Precio	Stock	Uso
3	Agua mineral 2L	$1400,00	40	C2: el precio se sube a $1600,00
2	Medialunas x6	$3200,00	3	C3: el borrador pide 5
11	Cerveza 1L	$4500,00	10	C4: se da de baja (`activo = false`)
9	Fideos secos 500g	$1950,00	30	C5: sin cambios
10	Helado 1L	$9500,00	0	C1/C7: ya dado de baja (no aparece en el catálogo)

Caso	Comando (resumen)	Resultado en `cola_sync`	Explicación
C2	Borrador: 2 × producto 3 visto a `140000`. En `bd2_tp8`: `UPDATE producto SET precio = 1600.00 WHERE id_producto = 3;` → instantánea → semáforo. Después: `UPDATE item_borrador SET precio_centavos_visto = 160000 …` + solicitud nueva `…000202` → semáforo.	`CONFLICTO` `precio cambió: producto 3 (visto 140000, actual 160000 centavos)` `SIMULADO_APROBADO` en la solicitud nueva	El precio del servidor no coincide con el que vio la tablet: amarillo, se le pregunta al cliente en vez de decidir solo. El cliente acepta, se actualiza el borrador y una solicitud con **otro UUID** (lo permite el índice parcial `ux_cola_una_pendiente`) da verde. El conflicto queda registrado.

Caso	Comando (resumen)	Resultado en `cola_sync`	Explicación
C3	Borrador: 5 × producto 2 (stock 3) → semáforo.	**RECHAZADO** `stock insuficiente: producto 2 (pide 5, hay 3)`	Pide más de lo que informa la instantánea: rojo. El stock del servidor sigue en 3: la tablet nunca lo descuenta.
C4	Borrador: 1 × producto 11. En `bd2_tp8`: `UPDATE producto SET activo = false WHERE id_producto = 11;` → instantánea → importar → semáforo.	**RECHAZADO** `dado de baja: producto 11` En `v_catalogo_disponible`: 0 filas · en `producto_cache`: la fila sigue con `eliminado = 1`	La baja viaja como lápida. Por el upsert, el producto sale del catálogo pero la fila no se borra, así el `item_borrador` que lo usa no rompe la clave foránea.
C5	Borrador: 2 × producto 9 a `195000` → semáforo.	**SIMULADO_APROBADO** `todo coincide con la instantánea (simulación, no es una venta)`	Activo, con stock (2 ≤ 30) y mismo precio: verde. Solo dice que pasaría las validaciones; la venta existiría recién cuando PostgreSQL la confirme en su propia transacción.
C6	(a) `.read movil/semaforo.sql` otra vez. (b) `INSERT INTO cola_sync` con `solicitud_uuid = …000005` (ya enviado).	(a) `quedan_pendientes = 0`, las 5 solicitudes sin cambios. (b) `UNIQUE constraint failed: cola_sync.solicitud_uuid`	La clave primaria impide duplicar el envío y el semáforo solo evalúa `PENDIENTE`, así que repetirlo no reevalúa nada.

> **C6 · Dónde va la protección real (parada 8):** en el servidor. Una tabla `solicitud_movil (solicitud_uuid PRIMARY KEY, …)` y, en la **misma transacción** que crea el pedido y descuenta stock (con `SELECT … FOR UPDATE`), un `INSERT … ON CONFLICT (solicitud_uuid) DO NOTHING`: si el UUID ya llegó por un reintento de red, no se crea otro pedido ni se descuenta stock dos veces. El TP no la implementa.

Consulta final en PostgreSQL: la simulación no cambió pedidos ni stock

```
-- psql -U postgres -d bd2_tp8 (antes y después)
SELECT count(*) FROM pedido;
SELECT id_producto, nombre, precio, stock, activo
FROM producto
WHERE id_producto IN (2, 3, 9, 10, 11)
ORDER BY id_producto;
```

Dato	Antes	Después
`count(*) FROM pedido`	200005	**200005**
Stock de 2 · 3 · 9 · 10 · 11	3 · 40 · 30 · 0 · 10	**3 · 40 · 30 · 0 · 10**
Precio del producto 3	1400.00	1600.00 *(C2, a mano)*
`activo` del producto 11	t	f *(C4, a mano)*

Pedidos y stock quedaron iguales. Los dos únicos cambios en `bd2_tp8` son los que se prepararon a mano para C2 y C4, antes de simular. Salidas completas en `evidencias/salida_casos_servidor_postgresql.txt` y `evidencias/salida_casos_c2_c6_sqlite.txt`.

Consigna 3.a · Mapa de la capa de datos

docs/2c_casos_y_3a_mapa.md

Motor	Qué guarda	Quién manda	Unidad
PostgreSQL `bd2_tp3` / `bd2_tp8`	Catálogo oficial (`producto`, `categoria`), `cliente`, `usuario`, `pedido` y `detalle_pedido` en 3FN.	**Fuente de verdad.** Es el único que decide precio vigente, stock real, altas y bajas del catálogo y confirmación de ventas.	1 a 6
MongoDB `foodstore.pedidos`	Pedidos ya cerrados como documentos: cliente y líneas embebidos, precio histórico congelado, `Decimal128`, validador `$jsonSchema` (8 pedidos del TP7).	Solo sobre esa copia de lectura de pedidos emitidos. **No decide stock, precios ni ventas nuevas.**	7
SQLite `foodstore_movil.db`	Copia fechada del catálogo (`snapshot_servidor`, `producto_cache` con lápidas), borradores y `cola_sync`. Sin datos personales.	**No manda sobre ningún dato maestro.** Solo prepara: muestra el catálogo, arma borradores y encola solicitudes.	8

PostgreSQL publica `v_snapshot_movil` (centavos, lápidas, sin datos personales) → ① CSV de la instantánea → **SQLite (tablet)** catálogo sin señal, borradores, semáforo local → ② solicitudes pendientes → ← ③ respuesta → **PostgreSQL** revalida stock (`FOR UPDATE`), precio e idempotencia y **solo él** confirma la venta → ④ pedidos cerrados → **MongoDB** lectura del pedido completo en un solo documento

En el TP, los pasos ② y ③ solo se simulan con el semáforo; ningún dato vuelve a escribirse en PostgreSQL desde la tablet.

Consigna 3.b · Plan de mantenimiento (agregado al plan de la U6)

docs/3b_plan_mantenimiento.md

¿Qué se respalda y cómo se prueba la restauración?

- **PostgreSQL:** respaldo diario con `backup_foodstore.sh` (`pg_dump -Fc` más un archivo `.conteo` con las filas de cada tabla, sin contraseñas en el script gracias a `pgpass.conf`). La restauración se prueba cada semana con `restore_test.sh` sobre la base descartable `foodstore_restore_test`: se mide el tiempo (RTO 15 min) y se compara el conteo contra el `.conteo` del respaldo.

- **MongoDB:** `mongodump --db foodstore` diario y prueba semanal con `mongorestore` en una base temporal, comparando la cantidad de documentos.
- **SQLite:** el catálogo de la tablet no se respalda porque se regenera desde PostgreSQL. Si al cerrar quedan solicitudes sin resolver en `cola_sync`, se copia el archivo `.db` (`.backup`) y se verifica con `PRAGMA integrity_check;`.

Si se restaura un respaldo viejo de PostgreSQL, ¿qué hacen las tablets? Quedan con un catálogo "del futuro" (precios o bajas que el respaldo todavía no tiene). Antes de vender o sincronizar, el servidor publica una instantánea nueva y **todas las tablets la descargan y recargan el caché** con `importar_catalogo.sql`. Las solicitudes resueltas después del corte se vuelven a evaluar contra el servidor restaurado; en un sistema real, la tabla `solicitud_movil` evita duplicar las que sí habían entrado al respaldo.

¿Cada cuánto y quién revisa `CONFLICTO` y `RECHAZADO`? Durante la feria, cada vez que vuelve la señal, el **vendedor** revisa con el cliente los `CONFLICTO` (acepta el precio nuevo y se reenvía, o se descarta el borrador) y corrige los `RECHAZADO` (quita o ajusta el ítem). Al cierre diario, antes del respaldo nocturno, el **encargado de operaciones** controla que no queden solicitudes sin cerrar y anota el motivo.

Consigna 3.c · Code review con IA (OpenCode, agente Plan)

docs/3c_code_review_ia.md · rama review-final

#	Unidad · archivo	Hallazgo	Decisión	Motivo
1	U3–U5 · `roles.sql`, `procedimientos.sql`	`rol_app_escritura` podía hacer `INSERT/UPDATE` directo en `pedido` salteando `registrar_pedido`.	Corregido	Se revocó el DML directo: solo se escribe con `CALL registrar_pedido`.
2	U3 · `procedimientos.sql`	Posible carrera al leer y descontar stock sin bloqueo explícito.	Aceptado	Documentado: la confirmación real de las tablets debe usar `SELECT … FOR UPDATE`.
3	U4 · `tp_desnormalizacion…sql`	Con `ON DELETE CASCADE`, el trigger de `detalle_pedido` ya no encuentra la fecha del pedido.	Corregido	Trigger `BEFORE DELETE ON pedido` que descuenta antes del borrado.
4	U5 · `funciones_auth.sql`	`fn_resetear_password` dejaba a soporte cambiar la clave de un `ADMIN`.	Corregido	`AND rol <> 'ADMIN'` y error si no encuentra el usuario.
5	U5 · `auditoria_test.sql`	Propuso `log_statement = 'all'` global y leer el log como UTF-8.	Descartado	Se dejó `'mod'`; el log en Windows es WIN1252.
6	U6 · `maintenance.sql`	`ANALYZE` omitía la tabla `usuario`.	Corregido	Desarrollo 2.
7	U6 · `restore_test.sh`	Comparaba la base restaurada contra la base viva.	Corregido	Desarrollo 2.
8	U6 · `health_check.sql`	Marcó como error `n_live_tup = 0` en una base copiada.	Descartado	`createdb -T` no copia esos contadores; el 0 era la señal útil.
9	U7 · `consultas_mongodb.js`	Índice único en `lineas.id_producto` y totales como `double`.	Descartado Corregido	Desarrollo 1; montos en `Decimal128`.
10	U8 · `schema_sqlite.sql`	Precio como texto en la vista; `UNIQUE(id_local)` total en `cola_sync`.	Descartado	Desarrollo 3.
11	U8 · `10_snapshot_catalogo.sql`, `semaforo.sql`	No miraba `categoria.activo`; control `CASE … 1/0`; sin casos borde.	Corregido Descartado	Baja por categoría; `1/0` falla siempre al planificar → `DO … RAISE`.

Desarrollo 1 · U7 (MongoDB) · Índice único en `lineas.id_producto` — Descartado

Propuesta: `db.pedidos.createIndex({ "lineas.id_producto": 1 }, { unique: true })` para que un producto no se repita dentro de un pedido. **Fundamento:** según la documentación de MongoDB, un índice único multikey impide que **dos documentos distintos** compartan el valor, no que se repita dentro del mismo arreglo. **Comprobación en mongosh** (`prueba_indice_unico_ia.js`): un pedido con dos líneas del producto 1 fue **aceptado**, y un segundo pedido legítimo con el producto 1 fue **rechazado** con `E11000 duplicate key`. Se usó en cambio un validador con `$expr` y `$setUnion`, que rechaza el duplicado con el código 121.

Desarrollo 2 · U6 (PostgreSQL) · `ANALYZE` sin `usuario` y restauración contra la base viva — Corregido

Hallazgo: `maintenance.sql` listaba 5 tablas y dejaba afuera `usuario` (creada en la U5); `restore_test.sh` comparaba la base restaurada contra `foodstore` en vivo, lo que da un falso error si entran ventas después del respaldo. **Comprobación en psql:** después del `ANALYZE`, `pg_class` mostraba `usuario` con `reltuples = -1` y `last_analyze` en `NULL`, mientras las otras 5 tablas estaban al día. Corregido: se analizan las 6 tablas (en la corrida siguiente `usuario` aparece analizada) y el respaldo guarda un `.conteo` contra el que valida la restauración.

Desarrollo 3 · U8 (SQLite) · Precio como texto y `UNIQUE` total en `cola_sync` — Descartado

Propuesta: cambiar `precio_centavos / 100.0` por `printf('%d.%02d', …)` en `v_catalogo_disponible` "para no usar REAL", y declarar `id_local UNIQUE` en `cola_sync`. **Fundamento:** la regla de centavos enteros es para **guardar** y comparar precios; la vista no guarda nada, y como texto `ORDER BY precio` ordena mal (`'899.00'` queda después de `'1500.00'`). Un `UNIQUE` total impediría reenviar el borrador del C2 después del conflicto. **Comprobación en sqlite3:** con el índice parcial `WHERE estado = 'PENDIENTE'`, dos pendientes del mismo borrador fallan con `UNIQUE constraint failed: cola_sync.id_local`, pero cuando el primero pasa a `CONFLICTO` la solicitud nueva entra sin error (es exactamente el C2).